



开放智慧助教

开发手册

基于教师个人知识库的智能语音交互教学系统

第一章 项目架构

1.1 整体架构

系统采用前后端一体架构（Next.js App Router），前端负责交互与状态管理，后端 API 路由集成 AI 能力。整体分为四层：

层次	技术	说明
用户层	React 19 + shadcn/ui	语音交互界面、管理面板、登录页
应用层	Next.js 16 API Routes	对话、记忆、知识库、教师管理等业务逻辑
AI 能力层	Coze SDK	LLM 对话、ASR 语音识别、TTS 语音合成、知识库检索、图片生成
数据层	Supabase + S3	教师档案、学生记忆、知识库、头像存储

1.2 目录结构

```
src/
├── app/
│   ├── api/
│   │   ├── audio/asr/route.ts      # 语音识别
│   │   ├── audio/tts/route.ts      # 语音合成
│   │   ├── audio/proxy/route.ts    # 音频代理 (CORS)
│   │   ├── chat/route.ts          # 智能对话 (核心)
│   │   ├── image/design/route.ts   # 图示设计
│   │   ├── image/generate/route.ts # 图片生成
│   │   ├── image/status/route.ts   # 异步任务状态
│   │   ├── knowledge/add/route.ts  # 添加知识库
│   │   ├── knowledge/upload/route.ts # 上传文件
│   │   ├── knowledge/search/route.ts # 搜索知识库
│   │   ├── memory/recall/route.ts  # 记忆检索
│   │   ├── memory/update/route.ts  # 记忆更新
│   │   ├── memory/profile/route.ts # 学生画像 CRUD
│   │   └── teacher/route.ts        # 教师管理
│   ├── page.tsx                    # 主页面 (数字教学)
│   ├── layout.tsx                  # 根布局
│   └── globals.css                  # 全局样式
├── components/
│   ├── DigitalHuman.tsx            # 数字人 (照片+音频波纹动画)
│   ├── VoiceSettings.tsx           # 声音设置
│   ├── KnowledgeManager.tsx        # 知识库管理+学习记忆
│   ├── StudentIdentity.tsx         # 学生身份管理
│   ├── TeacherDashboard.tsx        # 教师管理面板
│   ├── AdminDashboard.tsx          # 管理员后台
│   ├── LoginOverlay.tsx            # 登录页
│   └── ui/                          # shadcn/ui 组件库
├── lib/
│   ├── utils.ts                    # 工具函数
│   └── image-task-queue.ts         # 图片异步任务队列
└── storage/database/               # 数据库 Schema
```

1.3 请求流程

语音对话的完整请求流程：

- 1. 用户说话 → Web Audio API 采集 PCM → WAV 编码
- 2. WAV 发送至 /api/audio/asr → 返回识别文字
- 3. 文字+记忆上下文发送至 /api/chat → LLM 流式生成回答
- 4. 回答按句切分 → 逐句调用 /api/audio/tts → 预解码 AudioBuffer
- 5. AudioBuffer 队列顺序播放 → 音频波纹动画反馈
- 6. 对话结束 → 异步调用 /api/memory/update → LLM 提取记忆

第二章 技术栈与依赖

2.1 核心技术栈

类别	技术方案	版本	说明
前端框架	Next.js (App Router)	16	SSR+API 路由，前后端一体
核心库	React	19	组件化开发，Hooks 状态管理
语言	TypeScript	5	全栈类型安全
UI 组件	shadcn/ui + Radix UI	-	无障碍访问，可定制
样式	Tailwind CSS	4	原子化 CSS，vintage-grey 主题
大语言模型	豆包/DeepSeek	-	对话生成，知识检索
语音能力	Coze ASR/TTS	-	语音识别与合成
数据库	Supabase (PostgreSQL)	-	学生记忆持久化
对象存储	S3 兼容存储	-	教师头像管理

2.2 关键依赖

依赖包	用途
coze-coding-dev-sdk	AI 能力 SDK (LLM/ASR/TTS/知识库/图片生成/URL 解析)
@supabase/supabase-js	Supabase 客户端，数据库 CRUD
sonner	Toast 通知组件
docx	Word 文档生成 (报告导出)
lucide-react	图标库

第三章 核心模块详解

3.1 语音交互系统

语音交互是系统最核心的模块，基于 Web Audio API 构建持久化音频管道，实现实时语音对话。

音频管道架构

麦克风 → ScriptProcessorNode (PCM 采集)
→ GainNode (回声防护: speaking 时 gain=0)
→ AnalyserNode (VAD 音量检测 + 音频波纹动画)

TTS 播放: AudioBuffer 队列 → destination
→ AnalyserNode (音频波纹动画频谱数据)

关键机制

机制	实现方式	说明
回声防护	speaking 状态 gainNode.gain=0	TTS 播放时静音麦克风，杜绝回声被 ASR 识别
VAD 语音检测	音量阈值+校准+静音超时	首轮灵敏度优化: noiseFloor 初始值 3, 25 分位校准, 1.3x 阈值
TTS 预解码	fetch→decodeAudioData→队列	消除段间代理延迟，实现无缝播放
短句合并	<8 字暂不发送，等待累积	减少无效 TTS 请求，提升语音连贯性
手动打断	ttsSessionIdRef 递增	点击停止后飞行中 TTS 自动失效
音频波纹动画	AnalyserNode 频谱数据	TTS 播报期间实时分析音频频谱，驱动波纹动画
有序队列播放	ttsSeqRef/ttsNextPlayRef	确保语音严格按文字顺序播放

3.2 智能对话系统

对话流程

Chat API 是系统核心入口，接收用户消息后执行以下流程：

1. 并行加载：教师档案 + 学生记忆 + 知识库上下文
2. 拼接系统提示词：档案+价值观约束+记忆摘要+知识库
3. LLM 流式生成回答 (SSE 协议)
4. 同步返回文字流 → 异步触发 TTS
5. 流结束后异步调用 /api/memory/update

价值观领域约束

系统提示词中内置严格的领域约束规则，AI 助教只回答与教师专业相关的问题：

- 只回答教师专业领域内的问题，拒绝回答无关话题
- 禁止讨论政治、娱乐、宗教等非教学话题
- 偏离专业领域时礼貌引导回学习主题

【关键】价值观约束在系统提示词中设定，不受用户输入影响，确保教育安全。

并行化与缓存

- 教师档案、记忆检索、知识库查询从串行改为并行加载
- LRU 缓存：教师档案 (5min)、记忆摘要 (2min)、知识库检索 (3min)、TTS 音频 (30min)
- 首次对话延迟减少 200~500ms，重复查询零延迟

3.3 学生记忆系统

三阶段 workflow

阶段	时机	操作
对话前	Chat API 调用时	/api/memory/recall → 获取记忆摘要注入系统提示词
对话中	LLM 生成回答时	根据记忆个性化回复（已掌握不重复、薄弱重点讲）
对话后	流结束后异步	/api/memory/update → LLM 分析对话提取记忆信息

知识掌握三档评价

档位	mastery_level	教学策略
已掌握	≥ 0.6	不重复讲解基础概念，可进阶讨论，提供更深层次内容
学习中	0.3 ~ 0.5	有了解但未完全掌握，需巩固和补充，不需从零开始
薄弱	< 0.3	理解困难，需重点讲解，放慢节奏，用直观方式说明

掌握度评分机制

系统通过 LLM 分析学生对问题的回复，自动评价掌握程度并更新 mastery_level 数值：

评价类型	触发条件	mastery_level 变化	说明
mastered	学生通过验证测试（答对问题）	已有记录设为 0.8；新建时设为 0.7	必须通过测试验证，不能仅凭学生说懂了
practice	学生说懂了但未测试，或一般性练习	+0.1（上限 0.6，不超过已掌握阈值）	有进步但未经验证，不超过已掌握档位
confused	学生仍然困惑或答错	-0.1（下限 0）	降低掌握度，下次重点讲解

【验证测试机制】学生说"懂了"时，AI 必须出验证测试题，通过后才标记为已掌握。这是防止学生虚报掌握程度的关键机制。

重复记录合并

LLM 分析对话提取知识点时，可能产生重复记录。系统采用三级模糊匹配自动合并：

- 第一级：精确匹配（topic 完全相同）

- 第二级：subtopic 关键词匹配（关键词重叠）
- 第三级：topic+subtopic 全文关键词匹配
- 合并策略：保留最佳记录，合并 practice_count，删除重复记录

数据隔离

所有记忆数据按 teacher_id 隔离：同一学生在不同教师下有独立的画像、知识掌握、对话记录 and 教学策略。

3.4 图片生成

图片生成采用异步任务队列模式，避免前端长时间等待：

```
1. 前端提交 → /api/image/generate (async:true)
2. 立即返回 task_id
3. 后台执行：/api/image/design (LLM 生成 prompt)
   → Coze Image Generation (生成图片)
4. 前端轮询 /api/image/status?task_id=xxx → 获取结果
```

第四章 API 接口文档

4.1 接口总览

接口	方法	路径	功能
语音识别	POST	/api/audio/asr	语音转文字(WAV)
语音合成	POST	/api/audio/tts	文字转语音(多音色)
音频代理	GET	/api/audio/proxy	代理 TTS URL(CORS)
智能对话	POST	/api/chat	知识库问答(流式)
图示设计	POST	/api/image/design	LLM 生成图示 prompt
图片生成	POST	/api/image/generate	图片生成(同步/异步)
任务状态	GET	/api/image/status	异步任务状态查询
添加文档	POST	/api/knowledge/add	文本/URL 添加知识库
上传文件	POST	/api/knowledge/upload	文件上传 (.txt/.md/.csv/.json/.html/.xml/.docx/.pdf,

			FormData≤5MB)
搜索文档	POST	/api/knowledge/search	语义搜索知识库
记忆检索	POST	/api/memory/recall	对话前加载记忆上下文
记忆更新	POST	/api/memory/update	LLM 分析对话后写入记忆
学生画像	GET/POST/DELETE	/api/memory/profile	学生画像 CRUD
教师管理	GET/POST	/api/teacher	教师登录/档案/学生列表
教师头像	GET/POST	/api/teacher/avatar	头像上传/签名 URL
管理员	GET/POST	/api/admin	管理员登录/教师学生管理

4.2 核心 API 详解

POST /api/chat

智能对话核心接口，基于知识库和记忆系统生成个性化回答。

- 请求参数：message(用户消息)、teacher_id(教师 ID)、student_id(学生 ID)、session_id(会话 ID)
- 响应格式：SSE 流式输出(text/event-stream)
- 内部流程：并行加载→拼接提示词→LLM 流式生成→异步记忆更新
- 价值观约束：系统提示词严格限定 AI 只回答专业问题

POST /api/memory/recall

对话前检索学生记忆，生成三档分类摘要注入系统提示词。

- 请求参数：student_id、teacher_id
- 返回：学生画像、知识掌握（已掌握/学习中/薄弱三档）、近期对话摘要、教学策略
- 数据隔离：按 teacher_id 过滤，同一学生不同教师下有独立记忆

POST /api/memory/update

对话结束后由 LLM 自动分析对话内容，提取记忆信息写入数据库。

- 请求参数：student_id、teacher_id、对话内容
- LLM 分析维度：知识点掌握度(mastered/practice/confused)、教学策略、学生画像更新
- 自动合并：三级模糊匹配检测重复知识点，合并 practice_count 后删除重复记录

第五章 数据库设计

5.1 核心表

表名	说明	关键字段
teacher_profile	教师档案	id, name, password, subjects, expertise, teaching_style, avatar_key, knowledge_table, is_setup_complete, is_enabled
student_profile	学生画像	id, student_id, teacher_id, name, learning_style, interests, goals
knowledge_mastery	知识掌握追踪	id, student_id, teacher_id, topic, subtopic, mastery_level(0~1), practice_count, strong_points, weak_points
conversation_log	对话记录	id, student_id, teacher_id, session_id, role, content, timestamp
teaching_strategy	教学策略记忆	id, student_id, teacher_id, method_name, description, effectiveness
admin_account	管理员账号	id, username, password, created_at

5.2 知识库隔离

每位教师有独立的知识库表（表名=教师 ID），知识添加/搜索/对话都使用各自专属表，确保教师间数据完全隔离。

5.3 知识掌握评价核心字段

字段	类型	说明
mastery_level	FLOAT(0~1)	掌握度数值：≥0.6 已掌握，0.3~0.5 学习中，<0.3 薄弱
practice_count	INTEGER	练习次数，合并重复记录时累加
strong_points	TEXT	已掌握项的优势要点（验证测试通过后设置）
weak_points	TEXT	薄弱项/学习中的薄弱要点

第六章 AI 集成规范

6.1 SDK 使用规范

- coze-coding-dev-sdk 必须在后端 API 路由中使用，禁止客户端直接调用
- 所有 API 调用使用 HeaderUtils.extractForwardHeaders 转发请求头
- 对话 API 默认 SSE 流式输出（text/event-stream）
- 所有 API 调用包含 try-catch 错误处理

6.2 系统提示词结构

系统提示词组成：

1. 教师档案（姓名、科目、专业、教学风格）

2. 价值观领域约束（只回答专业问题，拒绝无关话题）

3. 记忆摘要（已掌握/学习中/薄弱三档，附交互规则）

4. 知识库检索结果（RAG 相关文档片段）

5. 交互规则（不重复已掌握、重点讲薄弱、验证测试等）

6.3 RAG 架构

知识库采用检索增强生成（RAG）架构：

- 教师上传文本/文件/URL → Coze Embedding 向量化 → 存入教师专属知识库表
- 对话时自动检索相关知识库内容 → 注入系统提示词 → LLM 结合知识生成回答
- 搜索 API 支持语义搜索，兼容旧表名 teacher_knowledge_base

6.4 ASR/TTS 集成

- ASR：接收 WAV 音频，调用 Coze ASR 转换文字，区分"无语音"和"格式错误"
- TTS：调用 Coze TTS 合成语音，speaker 白名单验证，失败自动回退默认音色
- 音频代理：/api/audio/proxy 解决 CORS 问题，支持 TTS URL 代理访问

6.5 LRU 缓存配置

缓存对象	TTL	说明
教师档案	5 分钟	避免每次对话查数据库
记忆摘要	2 分钟	同一学生短时间内复用
知识库检索	3 分钟	相似问题复用检索结果
TTS 音频	30 分钟	相同文本+音色复用语音

第七章 部署与运维

7.1 环境变量

变量名	说明	示例
COZE_WORKSPACE_PATH	项目工作目录	/workspace/projects/
COZE_PROJECT_DOMAIN_DEFAULT	对外访问域名	https://xxx.dev.coze.site
DEPLOY_RUN_PORT	服务监听端口	5000
COZE_PROJECT_ENV	运行环境	DEV / PROD

7.2 构建与启动

```
# 开发环境
coze dev                # 启动开发服务器(端口 5000, HMR 热更新)

# 生产环境
coze build              # 构建生产版本
coze start              # 启动生产服务器

# 包管理(仅 pnpm)
pnpm install            # 安装依赖
pnpm add <package>     # 添加依赖
```

7.3 数据库初始化

项目首次部署时, Supabase 数据库表和默认管理员账号会自动创建。默认管理员账号: admin / admin123。

7.4 开发规范

Ref 同步模式

为避免 useCallback 闭包捕获旧状态值, 所有跨回调共享的状态创建对应 ref, 渲染阶段同步赋值, 回调内部通过 ref.current 读取最新值。

Hydration 问题防范

- 严禁 JSX 中直接使用 typeof window、Date.now()、Math.random()
- 必须使用 use client + useEffect + useState 确保客户端渲染
- 禁止非法 HTML 嵌套 (如<p>嵌套<div>)

包管理约束

仅允许使用 pnpm 作为包管理器, 严禁使用 npm 或 yarn。